Recurrent Algorithms

Divide-and-conquer principle:
to divide a large problem into smaller ones and recursively solve each subproblem, then
to combine solutions of the subproblems to solve the original problem

Running time: by a ***recurrence relation*** combining the size and number of the subproblems and the cost of dividing the problem into the subproblems

“Telescoping” a Recurrence

Recurrence relation and its base condition (i.e., the difference equation and initial condition):

$$T(n) = 2 \cdot T(n-1) + 1;\ T(1) = 1$$

Closed (explicit) form for $T(n)$ by “telescoping”:

$$T(n) = 2\,T(n-1) + 1$$

$$T(n-1) = 2\,T(n-2) + 1$$

$$\ldots$$

$$T(2) = 2\,T(1) + 1$$

“Telescoping” ≡ Substitution

$$T(n) = 2T(n-1) + 1$$

$$2T(n-1) = 2^2 T(n-2) + 2$$

$$2^2 T(n-2) = 2^3 T(n-3) + 2^2$$

$$\ldots$$

$$2^{n-1} T(2) = 2^n T(1) + 2^{n-1}$$

$$T(n) = 1 + 2 + 2^2 + \ldots + 2^{n-1} + 2^n = 2^{n+1} - 1$$

Basic Recurrence: 1

Closed (explicit) form by “telescoping”:

$$T(n) = T(n-1) + n \quad \Leftrightarrow \quad T(n) = \frac{n(n+1)}{2}$$

$$T(n) = T(n-1) + n$$

$$T(n-1) = T(n-2) + n - 1$$

$$\dots$$

$$T(2) = T(1) + 2$$

$$T(1) = 1$$

1: Explicit Expression for $T(n)$

$$
\begin{aligned}
T(n) &= T(n-1) + n \\
&= \overline{T(n-2) + (n-1)} + n \\
&= \overline{T(n-3) + (n-2)} + (n-1) + n \\
\ldots &= \overline{T(2) + 3} + \ldots + (n-2) + (n-1) + n \\
&= \overline{T(1) + 2} + \ldots + (n-2) + (n-1) + n \\
&= \bar{1} + 2 + \ldots + (n-2) + (n-1) + n = \frac{n(n+1)}{2}
\end{aligned}
$$

Guessing to Solve a Recurrence

Infer (guess) a hypothetic solution $\mathrm{T}(n)$; $n \geq 0$ from a sequence of numbers $\mathrm{T}(0)$, $\mathrm{T}(1)$, $\mathrm{T}(2)$, …, obtained from the recurrence relation

Prove $\mathrm{T}(n)$ by math induction:

Base condition: T holds for $n = n_{\text{base}}$,e.g. $\mathrm{T}(0)$ or $\mathrm{T}(1)$

Induction hypothesis to verify: for every $n > n_{\text{base}}$, if T holds for $n - 1$, then T holds for n

Strong induction: if T holds for n_{base}, …, n - 1, then…

Explicit Expression for $T(n)$

$T(1) = 1; T(2) = 1 + 2 = 3; T(3) = 3 + 3 = 6;$

$T(4) = 6 + 4 = 10 \Rightarrow$ Hypothesis:

$$T(n) = \frac{n(n+1)}{2}$$

Base condition holds: $T(1) = 1 \cdot 2 / 2 = 1$

If the hypothetic closed-form relationship $T(n)$ holds for $n - 1$ then it holds also for n:

$$T(n) = T(n-1) + n = \frac{(n-1)n}{2} + n = \frac{n(n+1)}{2}$$

Thus, the expression for $T(n)$ holds for all $n > 1$

Basic Recurrence: 2

Repeated halving principle: halve the input in one step:

"Telescoping" (for $n = 2^m$):

$$T(n) = T(n/2) + 1 \quad \Leftrightarrow \quad T(n) \cong \log_2 n$$

$$\begin{array}{ll} T(2^m) = T(2^{m-1}) + 1 & T(2^2) = T(2^1) + 1 \\ T(2^{m-1}) = T(2^{m-2}) + 1 & T(2^1) = T(2^0) + 1 \\ \ldots & T(2^0) = 0 \end{array}$$

2: Explicit Expression for $T(n)$

$$
\begin{aligned}
T(2^m) &= T(2^{m-1}) + 1 \\
&= \overline{T(2^{m-2}) + 1} + 1 \\
&\ldots = \overline{T(2^1) + 1} + 1 + \ldots + 1 \\
&= \overline{T(2^0) + 1} + 1 + \ldots + 1 + 1 \\
T(2^m) &= m \quad \Rightarrow \quad T(n) = \log_2 n
\end{aligned}
$$

Basic Recurrence: 3

Scan and halve the input:

"Telescoping" (for $n = 2^m$): $\mathrm{T}(n) = \mathrm{T}(n/2) + n \quad \Leftrightarrow \quad \mathrm{T}(n) \cong 2n$

$$\begin{array}{l} \mathrm{T}(2^m) = \mathrm{T}(2^{m-1}) + 2^m \\ \mathrm{T}(2^{m-1}) = \mathrm{T}(2^{m-2}) + 2^{m-1} \\ \cdots \end{array} \qquad \begin{array}{l} \mathrm{T}(2^2) = \mathrm{T}(2^1) + 2^2 \\ \mathrm{T}(2^1) = \mathrm{T}(2^0) + 2^1 \\ \mathrm{T}(2^0) = 1 \end{array}$$

3: Explicit Expression for $\mathrm{T}(n)$

$$
\begin{aligned}
\mathrm{T}(2^m) &= \mathrm{T}(2^{m-1}) + 2^m \\
&= \overline{\mathrm{T}(2^{m-2}) + 2^{m-1}} + 2^m \\
\ldots &= \overline{\mathrm{T}(2^1) + 2^2} + \ldots + 2^{m-1} + 2^m \\
&= \overline{\mathrm{T}(2^0) + 2^1} + 2^2 + \ldots + 2^{m-1} + 2^m \\
\mathrm{T}(2^m) &= 2^{m+1} - 1 \;\Rightarrow\; \mathrm{T}(n) \cong 2n
\end{aligned}
$$

Basic Recurrence: 4

"Divide-and-conquer" prototype:

$$\mathrm{T}(n) = 2\,\mathrm{T}(n/2) + n \quad \Leftrightarrow \quad \mathrm{T}(n) \cong n \log_2 n$$

"Telescoping":

$$\frac{\mathrm{T}(n)}{n} = \frac{\mathrm{T}(n/2)}{n/2} + 1; \quad \mathrm{T}(1) = 0$$

For

$$n = 2^m \rightarrow \frac{\mathrm{T}(2^m)}{2^m} = \frac{\mathrm{T}(2^{m-1})}{2^{m-1}} + 1$$

4: Explicit Expression for $\mathrm{T}(n)$

$$
\begin{aligned}
\mathrm{T}(2^m)/2^m &= \mathrm{T}(2^{m-1})/2^{m-1} + 1 \\
&= \overline{\mathrm{T}(2^{m-2})/2^{m-2} + 1 + 1} \\
\ldots &= \overline{\mathrm{T}(2^1)/2^1 + 1 + \ldots + 1 + 1} \\
&= \overline{\mathrm{T}(2^0)/2^0 + 1 + 1 + \ldots + 1 + 1} \\
&= 0 + 1 + \ldots + 1 = m
\end{aligned}
$$

$$
\mathrm{T}(2^m) = m \cdot 2^m \quad \Rightarrow \quad \mathrm{T}(n) = n \log_2 n
$$

General "Divide-and-Conquer"

Theorem: The recurrence $\mathrm{T}(n) = a\,\mathrm{T}(n/b) + cn^k;\ \mathrm{T}(1) = c$

with integer constants $a \geq 1$ and $b \geq 2$ and positive constants c and k has the solution:

$$\mathrm{T}(n) = \begin{cases} O(n^{\log_b a}) & \text{if} \quad b^k < a \\ O(n^k \log n) & \text{if} \quad b^k = a \\ O(n^k) & \text{if} \quad b^k > a \end{cases}$$

Proof by telescoping: $n = b^m \Rightarrow \mathrm{T}(b^m) = a\,\mathrm{T}(b^{m-1}) + cb^{mk}$

General “Divide-and-Conquer”

Telescoping:

$$T(b^m) = aT(b^{m-1}) + cb^{mk}$$

$$aT(b^{m-1}) = a^2T(b^{m-2}) + acb^{(m-1)k}$$

$$a^2T(b^{m-2}) = a^3T(b^{m-3}) + acb^{(m-2)k}$$

$$\ldots \quad = \quad \ldots$$

$$a^{m-1}T(b) = a^mT(1) + a^{m-1}cb^k$$

$$\mathrm{T}(b^m) = a^mc + a^{m-1}cb^k + \ldots + acb^{(m-1)k} + cb^{mk}$$

$$= c\sum_{t=0}^{m} a^{m-t}b^{tk} = ca^m \sum_{t=0}^{m} \left(b^k/a\right)^t$$

Recursion Example

```
long factorial( int n )
{
   if( n <= 1 )
     return 1;
   else
     return n*factorial(n- 1);
}
```

In terms of big-Oh:

$t(1) = 1$

$t(n) = 1 + t(n-1) = 1 + 1 + t(n-2)$

$= \ldots k + t(n-k)$

Choose $k = n-1$

$t(n) = n-1 + t(1) = n-1 + 1 = O(n)$

Consider the following time complexity:

$t(0) = 1$

$t(n) = 1 + 2t(n-1) = 1 + 2(1 + 2t(n-2)) = 1 + 2 + 4t(n-2)$

$= 1 + 2 + 4(1 + 2t(n-3)) = 1 + 2 + 4 + 8t(n-3)$

$= 1 + 2 + \ldots + 2^{k-1} + 2^k t(n-k)$

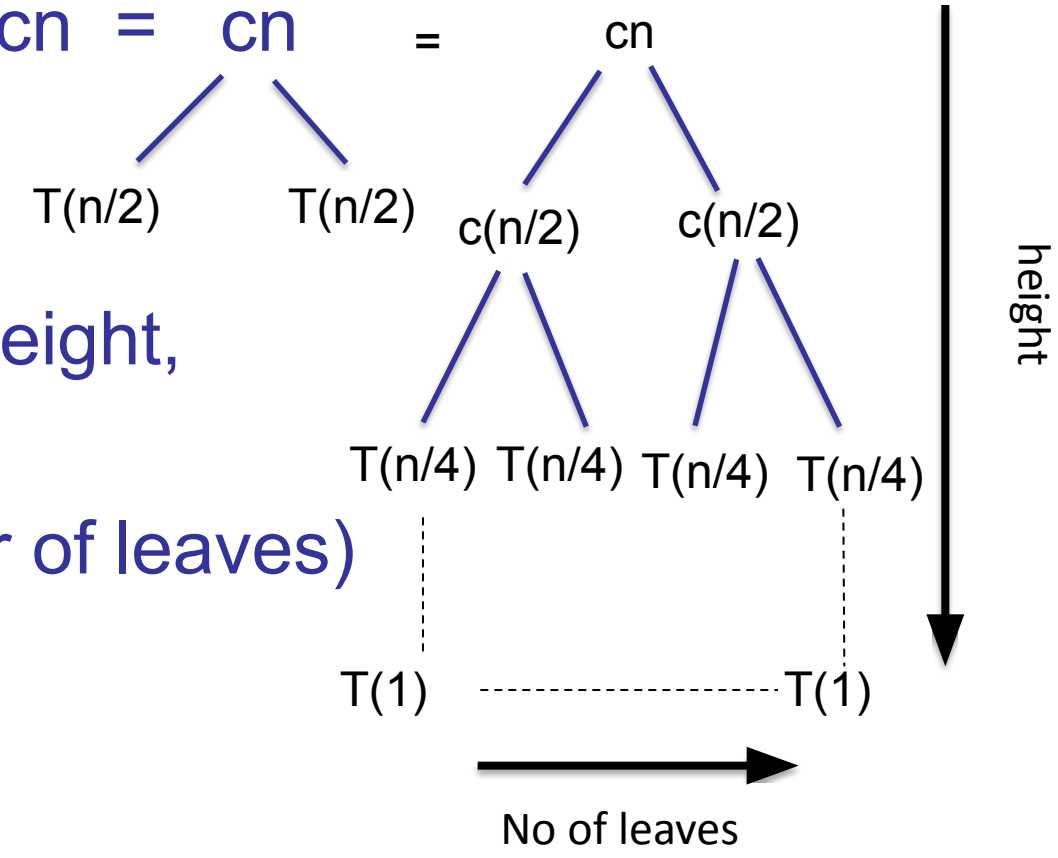
Choose $k = n$

$t(n) = 1 + 2 + \ldots 2^{n-1} + 2^n = 2^{n+1} - 1$

Another Approach (Recursive Tree Method)

- $T(n) = 2T(n/2) + O(n)$, $T(1) = O(1)$

$T(n) = T(n/2) + T(n/2) + cn = cn$



$T(n/2^k) = T(1)$ where k= height,

K= lgn

O(n) = O(height * number of leaves)

= O(k*n)=O(nlgn)

Another Example

- $T(n) = T(2n/3) + T(n/3) + O(n)$; $T(1) = 1$
- Two Sub-problems
- Different Sub-problem size (2n/3) and (n/3)
- Two different heights k_1 ,k_2 for (2n/3) and (n/3).
- $k_1 = \log_{3/2} n$, $k_2 = \log_3 n$
- Number of leaves is n
- Complexity ??
  - O(# of leaves * max (k_1,k_2)) ≈ O(n*max($\log_{3/2} n$, $\log_3 n$)) ≈ O(n$\log_{3/2} n$)≈ O(n$\log_2 n$)

Capabilities and Limitations

Rough complexity analysis cannot result immediately in an efficient practical program but it helps in predicting of empirical running time of the program

"Big-Oh" analysis is unsuitable for small input and hides the constants c and n_0 crucial for a practical task

"Big-Oh" analysis is unsuitable if costs of access to input data items vary and if there is lack of sufficient memory

But complexity analysis provides ideas how to develop new efficient methods